Consistency Models

linearizable vs causal vs eventual, and session guarantees

Watch the 3-min video

In one line: consistency is a spectrum, from linearizable (one copy, costly) to eventual (cheap, may be stale); session guarantees like read-your-writes protect each user in between.

1 The spectrum

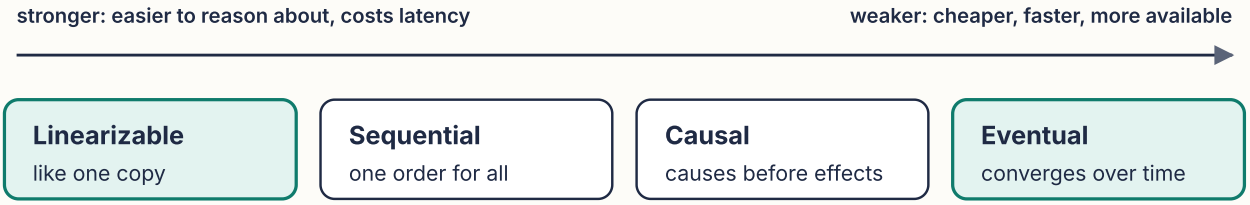


Figure 1. Stronger models are easier to reason about but need coordination; weaker ones are cheaper and stay available.

Model	Promise	Typical use
Linearizable	Once a write completes, every later read (by anyone) sees it, as if there were one copy	Locks, leader election, balances
Sequential	Everyone sees operations in the same order, but possibly behind real time	Replicated logs
Causal	If A caused B (a reply to a post), everyone sees A before B; unrelated events may appear in different orders	Comments, chat threads
Eventual	If writes stop, replicas converge; until then reads may be stale	Likes, view counts, feeds

2 Session guarantees

Between the extremes, session guarantees protect a single user's experience:

- **Read-your-writes:** you always see your own changes.
- **Monotonic reads:** once you've seen a newer value, you never see an older one again.
- **Monotonic writes:** your writes are applied in the order you made them.
- **Writes-follow-reads:** a reply you write is ordered after the post you read.

3 Why read-your-writes matters

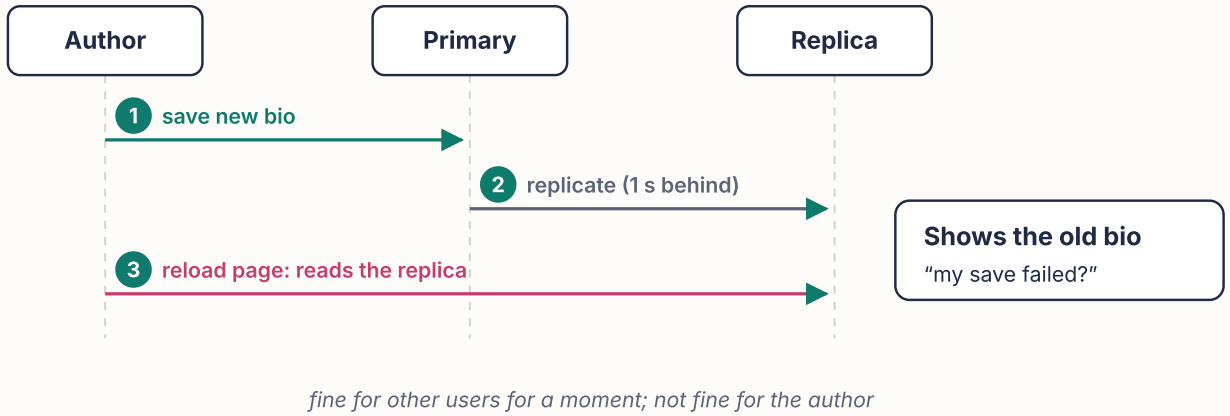


Figure 2. The write goes to the primary, the reload reads a lagging replica, and the author sees the old value.

THREE WAYS TO PROVIDE IT ON REPLICAS

1. **Read your own data from the primary** for a short window (e.g. 10 s) after you write.
2. **Track a version:** remember the log position (LSN) of the user's last write in the session, and read only from a replica that has caught up to it.
3. **Pin the user to one replica** to get monotonic reads, so time never goes backwards for them.

COMMON MISTAKES

- Treating consistency as a yes/no choice.
- Using eventual consistency for data the author just edited.
- Load balancing reads across replicas so a user sees values jump back and forth.
- Paying for linearizability where causal or eventual is enough.

4 Key terms

Linearizable: behaves like one copy

Eventual: replicas converge over time

Monotonic reads: never see an older value again

Causal: causes are seen before effects

Read-your-writes: you see your own changes

Replica lag: how far behind a replica is

5 Interview questions

Q1. Linearizability vs eventual consistency? BASIC

Linearizable: every read sees the latest completed write, as if there were one copy. Eventual: replicas converge only over time, so reads may be stale.

Q2. What is causal consistency? BASIC

Operations that are causally related (a reply and its post) are seen in the same order by everyone; unrelated operations may be seen in different orders.

Q3. How do you provide read-your-writes on replicas? BASIC

Route the author's reads to the primary for a short window, or read only from replicas that have caught up to the version of their last write.

Q4. Comments sometimes appear before the post they reply to. Which model fixes it? SCENARIO

Causal consistency: track the dependency (the reply references the post's version) and don't show the reply until the post is visible on that replica.

Q5. A user's balance flickers between two values on refresh. Why, and how do you fix it? SCENARIO

Reads hit different replicas with different lag. Provide monotonic reads by pinning the user to one replica, or read balances from the primary.

Q6. Which consistency model for a like counter, and for a seat reservation? SCENARIO

Likes: eventual (a few seconds stale is harmless). Seat reservation: linearizable (two people must never get the same seat).

Q7. Does eventual consistency mean data can be lost? TRICKY

No. It's about when replicas agree, not durability. An acknowledged write is still stored; some replicas just see it later.

Q8. Is linearizable the same as serializable? TRICKY

No. Linearizability is about single-object reads and writes being up to date in real time. Serializability is about transactions over many objects behaving as if run one at a time. Strict serializability is both.

Next: SD-A-016 · Fault Tolerance backendroadmap.com